

实战：ChatGPT Plugin 开发

彭靖田

Google Developer Experts
LangChain Developer



目录

- ChatGPT Plugin 开发指南
 - ChatGPT Plugin 介绍
 - 样例项目：待办（Todo）管理插件
 - 实战样例部署与测试
 - ChatGPT 开发者模式
- 实战：天气预报（Weather Forecast）插件开发
 - Weather Forecast Plugin 设计与定义
 - 天气预报函数服务化
 - 第三方天气查询平台对接
 - 实战 Weather Forecast Plugin
- Function Calling vs ChatGPT plugin

ChatGPT Plugin 开发指南

ChatGPT plugins

We've implemented initial support for plugins in ChatGPT. Plugins are tools designed specifically for language models with safety as a core principle, and help ChatGPT access up-to-date information, run computations, or use third-party services.

[Join plugins waitlist](#)

[Read documentation ↗](#)

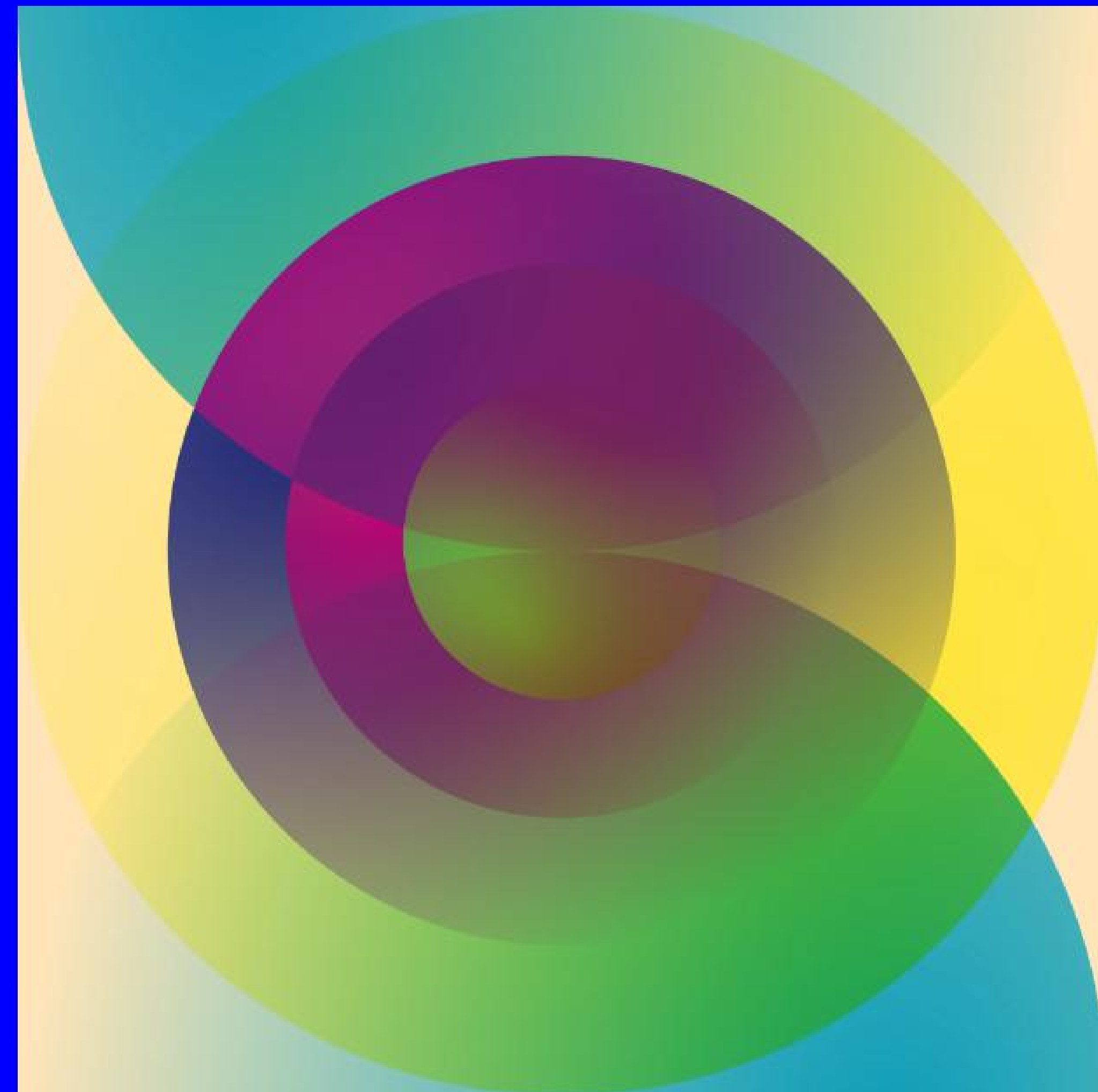
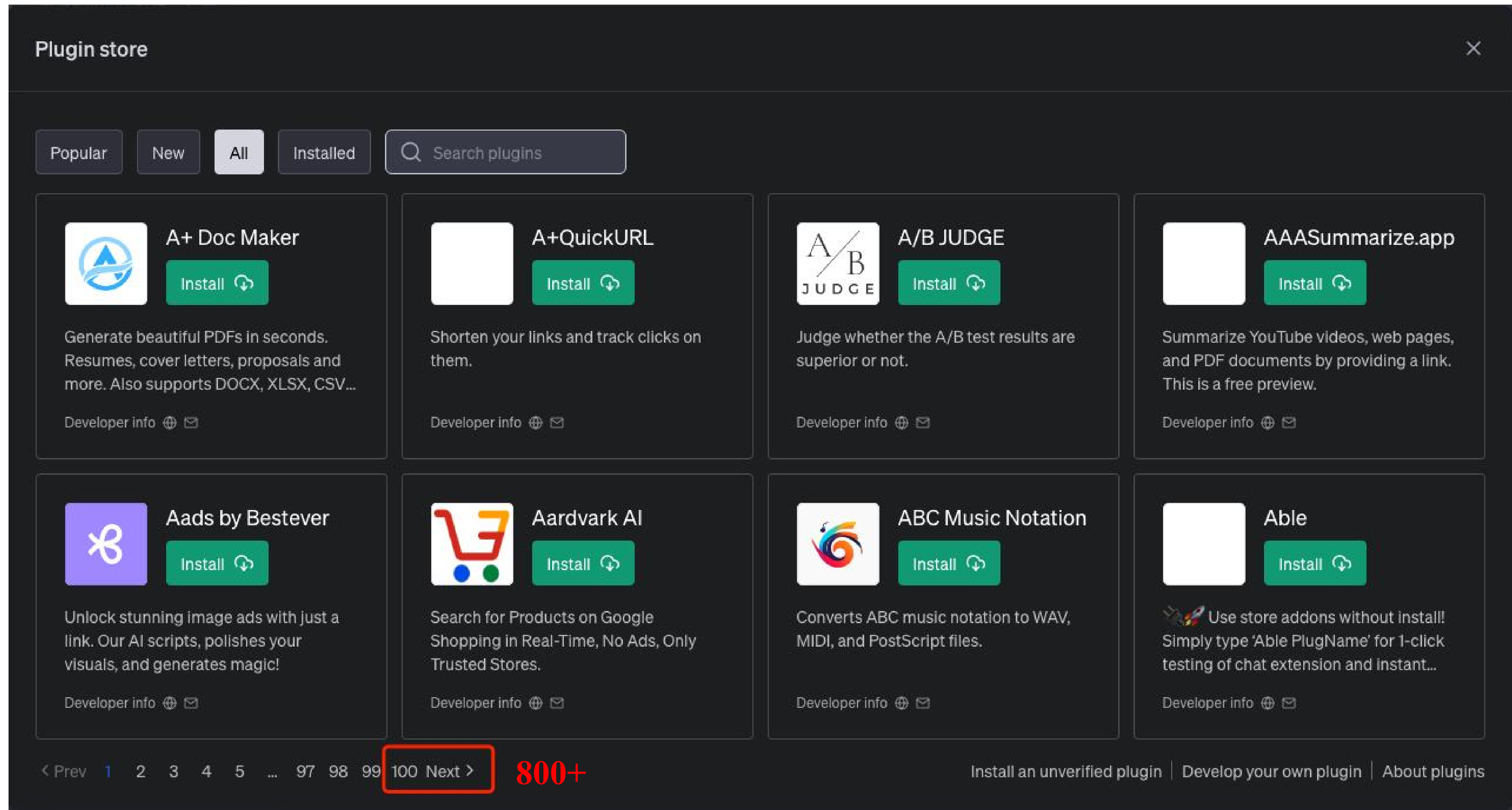


Illustration: Ruby Chen



愿景： 基于 ChatGPT 能力， 以Plugin形式， 赋能千行百业



 Expedia Bring your trip plans to life—get there, stay there, find things to see and do.	 FiscalNote Provides and enables access to select market-leading, real-time data sets for legal, political, and regulatory data and information.	 Instacart Order from your favorite local grocery stores.	 KAYAK Search for flights, stays and rental cars. Get recommendations for all the places you can go within your budget.
 Klarna Shopping Search and compare prices from thousands of online shops.	 Milo Family AI Giving parents superpowers to turn the manic to magic, 20 minutes each day. Ask: Hey Milo, what's magic today?	 OpenTable Provides restaurant recommendations, with a direct link to book.	 Shop Search for millions of products from the world's greatest brands.
 Speak Learn how to say anything in another language with Speak, your AI-powered language tutor.	 Wolfram Access computation, math, curated knowledge & real-time data through Wolfram Alpha and Wolfram Language.	 Zapier Interact with over 5,000+ apps like Google Sheets, Trello, Gmail, HubSpot, Salesforce, and more.	

开发 ChatGPT Plugin 难么？

样例项目：待办（Todo）管理插件



Plugin flow

To build a plugin, it is important to understand the end-to-end flow.

- 1 Create a manifest file and host it at `yourdomain.com/.well-known/ai-plugin.json`
 - The file includes metadata about your plugin (name, logo, etc.), details about authentication required (type of auth, OAuth URLs, etc.), and an OpenAPI spec for the endpoints you want to expose.
 - The model will see the OpenAPI description fields, which can be used to provide a natural language description for the different fields.
 - We suggest exposing only 1-2 endpoints in the beginning with a minimum number of parameters to minimize the length of the text. The plugin description, API requests, and API responses are all inserted into the conversation with ChatGPT. This counts against the context limit of the model.
- 2 Register your plugin in the [ChatGPT UI](#)
 - Select the plugin model from the top drop down, then select “Plugins”, “Plugin Store”, and finally “Develop your own plugin”.
 - If authentication is required, provide an OAuth 2 `client_id` and `client_secret` or an API key.
- 3 Users activate your plugin
 - Users must manually activate your plugin in the ChatGPT UI. (ChatGPT will not use your plugin by default.)
 - You will be able to share your plugin with 100 additional users (only other developers can install unverified plugins).
 - If OAuth is required, users will be redirected via OAuth to your plugin to sign in.
- 4 Users begin a conversation
 - OpenAI will inject a compact description of your plugin in a message to ChatGPT, invisible to end users. This will include the plugin description, endpoints, and examples.
 - When a user asks a relevant question, the model may choose to invoke an API call from your plugin if it seems relevant; for POST requests, we require that developers build a user confirmation flow to avoid destruction actions.
 - The model will incorporate the API call results into its response to the user.
 - The model might include links returned from the API calls in its response. These will be displayed as rich previews (using the [OpenGraph protocol](#), where we pull the `site_name`, `title`, `description`, `image`, and `url` fields).
 - The model can also format data from your API in markdown and the ChatGPT UI will render the markdown automatically.

ChatGPT Plugin 核心概念与文件

OpenAPI定义(openapi.yaml)

```
1 openapi: 3.0.1
2 info:
3   title: TODO Plugin
4   description: A plugin that allows the user to create and manage todos
5   version: 'v1'
6 servers:
7   - url: http://localhost:5003
8 paths:
9   /todos/{username}:
10    get:
11      operationId: getTodos
12      summary: Get the list of todos
13      parameters:
14        - in: path
15          name: username
16          schema:
17            type: string
18            required: true
19            description: The name of the user.
20      responses:
21        "200":
22          description: OK
23          content:
24            application/json:
25              schema:
26                $ref: '#/components/schemas/getTodosResponse'
27 >
28 >
29 >
30 >
31 >
32 >
33 >
34 >
35 >
36 >
37 >
38 >
39 >
40 >
41 >
42 >
43 >
44 >
45 >
46 >
47 >
48 >
49 >
50 >
51 >
52 >
53 >
54 >
55 >
56 >
57 >
58 >
59 >
60 >
61 >
62 >
63 >
64 >
65 >
66 components:
67   schemas:
68     getTodosResponse:
69       type: object
70       properties:
71         todos:
72           type: array
73           items:
```

插件元数据定义(ai-plugin.json)

```
gins > todo-list > .well-known > {} ai-plu
"schema_version": "v1",
"name_for_human": "TODO List (no a
"name_for_model": "todo",
"description_for_human": "Manage y
"description_for_model": "Plugin f
"auth": {
  "type": "none"
},
"api": {
  "type": "openapi",
  "url": "http://localhost:5003/op
},
"logo_url": "http://localhost:5003
"contact_email": "legal@example.co
"legal_info_url": "http://example.
```

服务接口定义(xx.py)

```
chatgpt-plugins > todo-list > main.py > ...
1 import json
2
3 import quart
4 import quart_cors
5 from quart import request
6
7 app = quart_cors.cors(quart.Quart(__name__), allow_origin="https
8
9 # Keep track of todo's. Does not persist if Python session is re
10 _TODOS = {}
11
12 @app.post("/todos/<string:username>")
13 async def add_todo(username):
14     request = await quart.request.get_json(force=True)
15     if username not in _TODOS:
16         _TODOS[username] = []
17     _TODOS[username].append(request["todo"])
18     return quart.Response(response='OK', status=200)
19
20 @app.get("/todos/<string:username>")
21 async def get_todos(username):
22     return quart.Response(response=json.dumps(_TODOS.get(username)
23
24 @app.delete("/todos/<string:username>")
25 async def delete_todo(username):
26     request = await quart.request.get_json(force=True)
27     todo_idx = request["todo_idx"]
28     # fail silently, it's a simple plugin
29     if 0 <= todo_idx < len(_TODOS[username]):
30         _TODOS[username].pop(todo_idx)
31     return quart.Response(response='OK', status=200)
32
33 @app.get("/logo.png")
34 async def plugin_logo():
35     filename = 'logo.png'
36     return await quart.send_file(filename, mimetype='image/png')
37
```


DEMO

实战：天气预报 (**Weather Forecast**) 插件开发

Weather Forecast Plugin: ai-plugin.json

```
chatgpt-plugins > weather-forecast > .well-known > {} ai-plugin.json > ...
```

```
1  {
2    "schema_version": "v1",
3    "name_for_human": "Weather Forecast",
4    "name_for_model": "weather",
5    "description_for_human": "Global Weather Forecast. You can ask the current or future weather of any city around the world.",
6    "description_for_model": "Plugin for managing weather forecasts. Search current weather and future forecasts through provided api.",
7    "auth": {
8      "type": "none"
9    },
10   "api": {
11     "type": "openapi",
12     "url": "http://localhost:5002/openapi.yaml"
13   },
14   "logo_url": "http://localhost:5002/logo.png",
15   "contact_email": "pjt73651@gmail.com",
16   "legal_info_url": "http://example.com/legal"
17 }
18
```


Weather Forecast Plugin: openapi.yaml

chatgpt-plugins > weather-forecast > ! openapi.yaml

```

1  openapi: 3.0.1
2  info:
3    title: Weather Forecast
4    description: A Plugin that allows the user to forecast current or f
5    version: 'v1'
6  servers:
7    - url: http://localhost:5002
8  paths:
9    /weather/current:
10     get:
11       operationId: getCurrentWeather
12       summary: Get the current weather of the city
13       parameters:
14         - in: query
15           name: city
16           schema:
17             type: string
18             required: true
19             description: The city and state, e.g. San Francisco, CA.
20       responses:
21         "200":
22           description: OK
23           content:
24             application/json:
25               schema:
26                 $ref: '#/components/schemas/getCurrentWeather'

```

```

/weather/forecast:
  get:
    operationId: getNDayWeatherForecast
    summary: Forecast the weather of the city in a few days
    parameters:
      - in: query
        name: num_days
        schema:
          type: integer
          required: true
          description: The number of days to forecast, e.g. 5
      - in: query
        name: city
        schema:
          type: string
          required: true
          description: The city and state, e.g. San Francisco, CA.
    responses:
      "200":
        description: OK
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/getNDayWeatherForecast'
components:
  schemas:
    getCurrentWeather:
      type: object
      properties:
        weather:
          type: string
          description: The current weather of the city.
    getNDayWeatherForecast:
      type: object
      properties:
        weather:
          type: string
          description: The weather of the city in a few days.

```


Weather Forecast Plugin: main.py

```
# 定义一个路由处理器, 处理 GET 请求, 路由为 "/logo.png"
@app.get("/logo.png")
async def plugin_logo():
    # 文件名为 'weather-forecast.png'
    filename = 'weather-forecast.png'
    # 使用 quart.send_file 方法发送文件, 并指定 MIME 类型为 'image/png'
    return await quart.send_file(filename, mimetype='image/png')

# 定义一个路由处理器, 处理 GET 请求, 路由为 "/.well-known/ai-plugin.json"
@app.get("/.well-known/ai-plugin.json")
async def plugin_manifest():
    # 从请求头中获取主机名
    host = request.headers['Host']
    # 打开并读取 ai-plugin.json 文件
    with open("./.well-known/ai-plugin.json") as f:
        text = f.read()
    # 返回一个 Quart 响应对象, 响应体为 text, MIME 类型为 "text/json"
    return quart.Response(text, mimetype="text/json")

# 定义一个路由处理器, 处理 GET 请求, 路由为 "/openapi.yaml"
@app.get("/openapi.yaml")
async def openapi_spec():
    # 从请求头中获取主机名
    host = request.headers['Host']
    # 打开并读取 openapi.yaml 文件
    with open("openapi.yaml") as f:
        text = f.read()
    # 返回一个 Quart 响应对象, 响应体为 text, MIME 类型为 "text/yaml"
    return quart.Response(text, mimetype="text/yaml")
```

```
# 定义一个路由处理器, 处理 GET 请求, 路由为 "/weather/current"
@app.get("/weather/current")
async def get_current_weather():
    # 从请求参数中获取城市名
    city = request.args.get("city")

    response = _get_current_weather(city)
    # 创建并返回一个 Quart 响应对象, 状态码为 200, 响应体为 response 的 json 形式
    return quart.Response(json.dumps(response), status=200)

# 定义一个路由处理器, 处理 GET 请求, 路由为 "/weather/forecast"
@app.get("/weather/forecast")
async def get_n_day_weather_forecast():
    # 从请求参数中获取城市名和预报天数
    city = request.args.get("city")
    num_days = int(request.args.get("num_days"))

    response = _get_n_day_weather_forecast(city, num_days)
    # 创建并返回一个 Quart 响应对象, 状态码为 200, 响应体为 response 的 json 形式
    return quart.Response(json.dumps(response), status=200)
```


Quart : A Fast Python Web Microframework

Quart is a Fast Python web microframework. Using Quart you can,

- write JSON APIs e.g. [a RESTful API](#),
- render and serve HTML e.g. [a blog](#),
- serve WebSockets e.g. [a simple chat](#),
- stream responses e.g. [serve video](#),
- all of the above in a single app,
- or do pretty much anything over the HTTP or WebSocket protocols.



第三方天气
查询平台

[返回旧版文档](#)

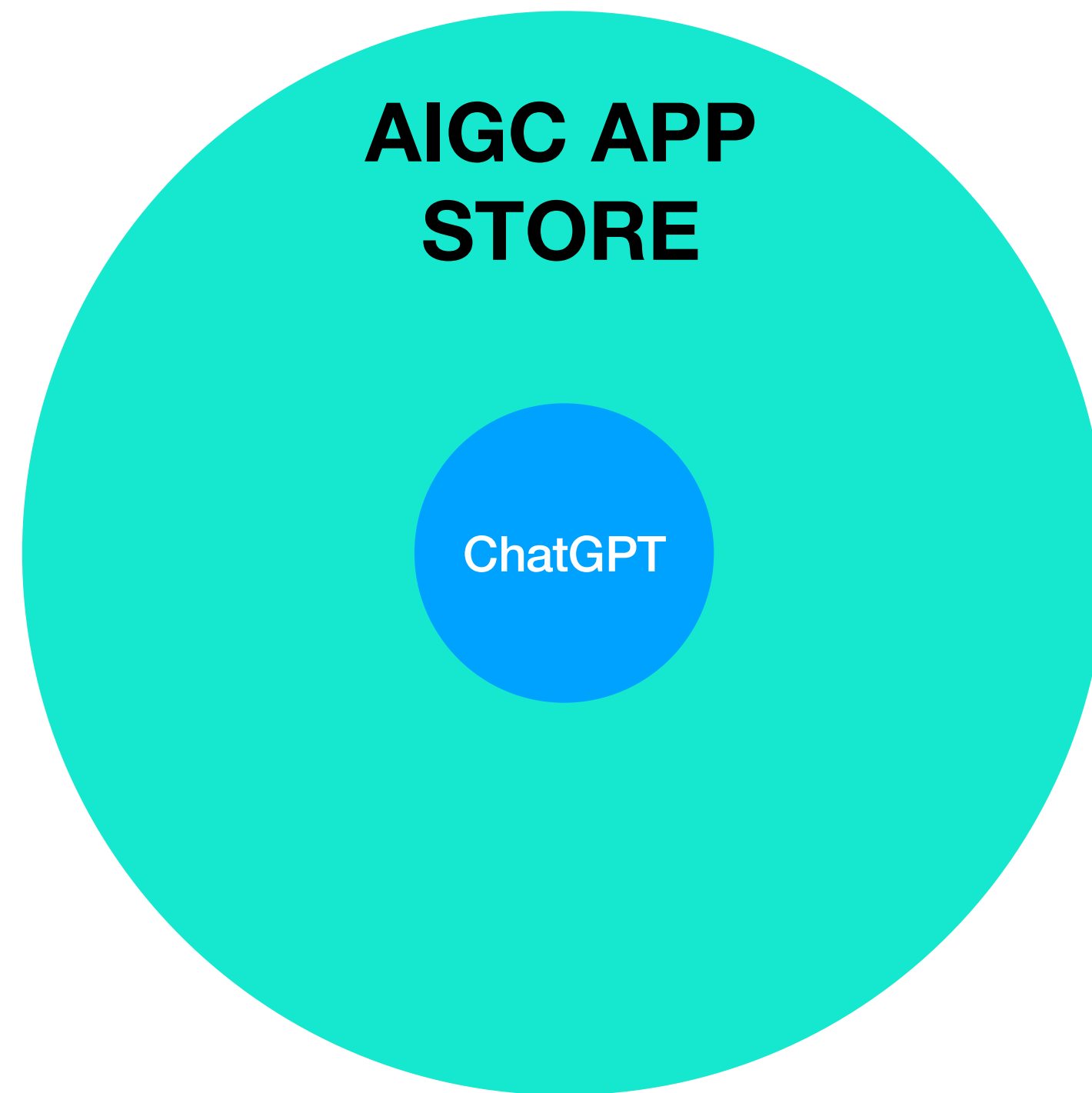
轨迹纠偏

<https://lbs.amap.com/api/webservice/guide/api/weatherinfo>

DEMO

Function Calling vs ChatGPT plugin

Function Calling vs ChatGPT plugin



课程项目：GitHub openai-quickstart

openai-quickstartPublic

UnpinUnwatch10Fork145Starred306

main1 branch0 tagsGo to fileAdd fileCode

DjangoPeng Merge branch 'main' of github.com:DjangoPeng/openai-quickstart int...6bd7e132 minutes ago57 commits

chatgpt-plugins	add initial version of chatgpt plugins	2 minutes ago
docs	make layout more readable	last week
openai-translator	Merge branch 'main' of github.com:DjangoPeng/openai-quickstart in...	2 minutes ago
openai_api	Remove fine_food_reviews_with_embeddings_1k_demo.csv	last week
.gitignore	update embedding demo	2 weeks ago
LICENSE	Initial commit	2 weeks ago
README-CN.md	fix: correct file path of pdfplumber.ipynb on readme	yesterday
README.md	fix: correct file path of pdfplumber.ipynb on readme	yesterday

README.md

OpenAI Quickstart

English | 中文

This project is designed as a one-stop learning resource for anyone interested in large language models and their application in Artificial Intelligence Governance and Control (AIGC) scenarios. By providing theoretical foundations, development basics, and hands-on examples, this project offers comprehensive guidance on these cutting-edge topics.

Features

About

A comprehensive guide to understanding and implementing large language models with hands-on examples using LangChain for AIGC applications.

- Readme
- Apache-2.0 license
- Activity
- 306 stars
- 10 watching
- 145 forks

Releases

No releases published
[Create a new release](#)

Packages

No packages published
[Publish your first package](#)

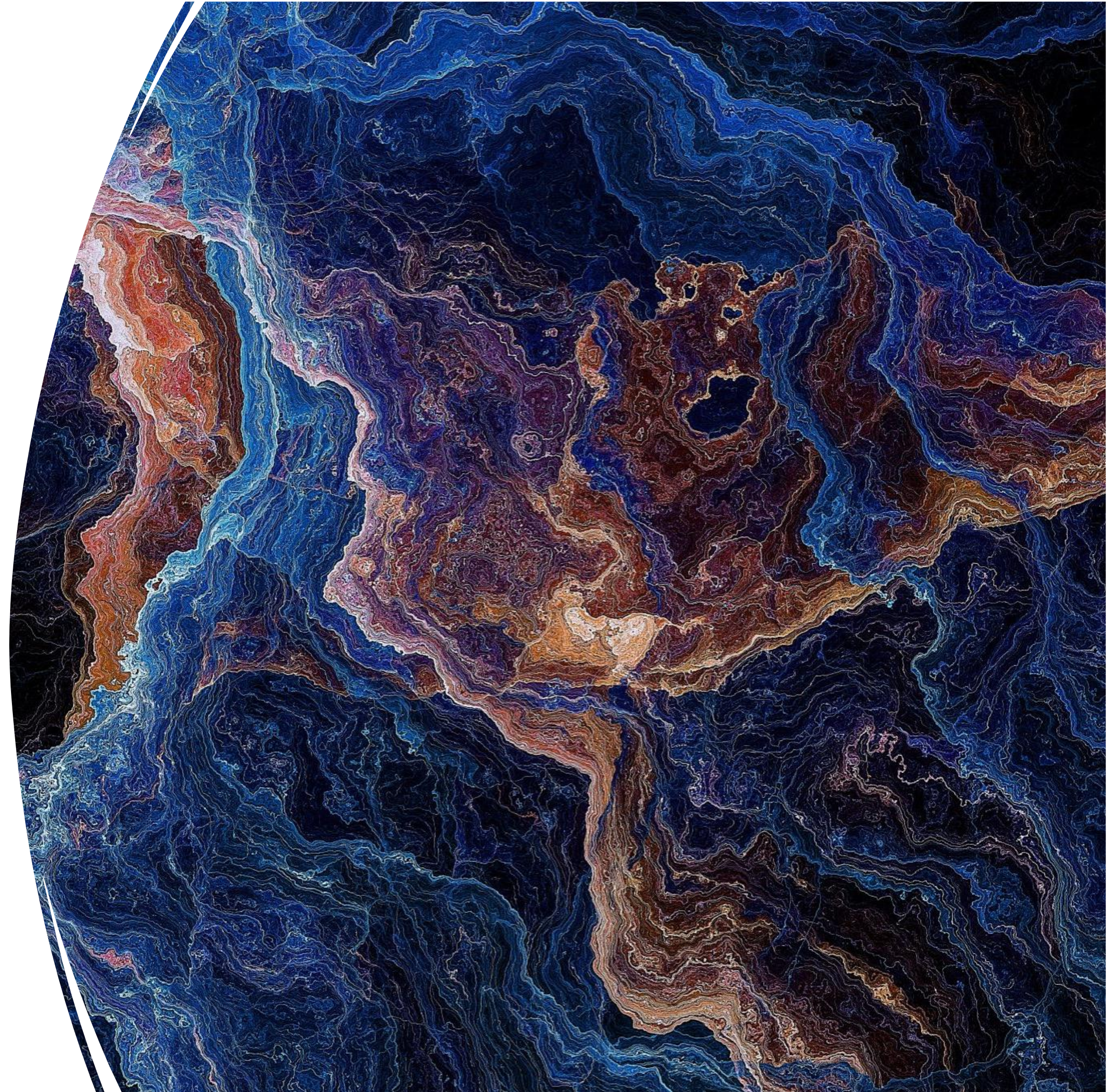
Contributors8

<https://github.com/DjangoPeng/openai-quickstart>

产品规划v2.0

Feat List

- 支持图形用户界面 (GUI)，提升易用性。
- 添加对保留源 PDF 的原始布局的支持。
- 服务化：以API形式提供翻译服务支持。
- 添加对其他语言的支持。



有奖活动

将完成此次实战作业的关键步骤、方法及心得体会整理成一篇学习笔记，发表在指定的平台，将有机会获得极客时间送出的精美礼品。

奖品设置

- 一等奖：500 元现金（5名）
- 二等奖：极客时间精美伴手礼（30名）
- 三等奖：200元训练营复购优惠券（200名）

具体活动规则班主任周一会在微信群内向大家公布



伴手礼盒